MODULO 18

Complejidad temporal y espacial (Big O)

Resumen de consulta rapida para C++

Objetivo: Big O describe como crece el trabajo o la memoria de un algoritmo cuando aumenta la cantidad de datos. No mide segundos exactos.

1. ¿Que es Big O?

Big O sirve para describir como crece el trabajo de un algoritmo cuando aumenta la cantidad de datos. La variable n normalmente representa la cantidad de elementos de entrada.

```
for (int numero : numeros)
{
   cout << numero << endl;
}
```

Si hay n elementos, el ciclo hace n iteraciones. Su complejidad es O(n).

2. O(1) - Tiempo constante

El trabajo no aumenta aunque aumente el tamaño de los datos.

```
cout << numeros[5];
```

Acceder directamente a una posicion de un vector es O(1). Da igual si el vector tiene 10 o 1,000,000 de elementos.

3. O(n) - Tiempo lineal

El trabajo crece proporcionalmente a la cantidad de elementos.

```
for (int numero : numeros)
{
   cout << numero << endl;
}
```

Con 10 elementos hay aproximadamente 10 iteraciones; con 1000, aproximadamente 1000.

4. Las constantes se eliminan

Dos recorridos separados producen O(n + n) = O(2n), pero Big O lo simplifica a O(n).

```
for (...) { }
for (...) { }
```

O(2n), O(5n) y O(100n) se simplifican a O(n).

5. O(n^2) - Tiempo cuadratico

Suele aparecer con dos ciclos anidados.

```
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
   }
}
```

n x n = n^2, por lo que la complejidad es O(n^2).

6. O(n^3)

Tres ciclos anidados que dependen de n producen crecimiento cubico.

```
for (...)
{
   for (...)
   {
      for (...)
      {
      }
   }
}
```

n x n x n = n^3, por lo que la complejidad es O(n^3).

7. Ciclos separados vs ciclos anidados

Los ciclos separados se suman; los ciclos anidados se multiplican.

```
Separados: O(n) + O(n) = O(n)
Anidados: O(n) x O(n) = O(n^2)
```

Regla: separados -> suma; anidados -> multiplicacion.

8. Termino dominante

Cuando hay varios terminos, nos quedamos con el que crece mas rapido.

```
O(n^2 + n) -> O(n^2)
O(n^3 + n^2 + n) -> O(n^3)
O(n + 1) -> O(n)
```

El termino de mayor crecimiento domina para valores grandes de n.

9. O(log n) - Tiempo logaritmico

Aparece cuando el problema se reduce mucho en cada paso, normalmente a la mitad.

```
while (n > 1)
{
   n /= 2;
}
```

Ejemplo: 1024 -> 512 -> 256 -> ... -> 1. Solo requiere unas 10 reducciones.

10. O(n log n)

Aparece cuando se realiza un trabajo logaritmico n veces.

```
for (int i = 0; i < n; i++)
{
   int numero = n;
   while (numero > 1)
   {
      numero /= 2;
   }
}
```

O(n) x O(log n) = O(n log n). Es comun en ordenamientos eficientes.

11. Orden de complejidades

De mejor a peor escalabilidad:

```
O(1)
O(log n)
O(n)
O(n log n)
O(n^2)
O(n^3)
```

Idea clave: Mientras mas abajo este una complejidad en esta lista, mas rapido crece el trabajo cuando aumenta n.

12. Mejor caso y peor caso

Un mismo algoritmo puede hacer diferente cantidad de trabajo dependiendo de los datos.

```
for (int numero : numeros)
{
   if (numero == buscado)
   {
      break;
   }
}
```

- Mejor caso: O(1) si el elemento aparece primero.
- Peor caso: O(n) si esta al final o no existe.

13. Complejidad espacial

Mide cuanta memoria adicional necesita un algoritmo conforme aumenta n. No es lo mismo que complejidad temporal.

14. O(1) espacio

```
int mayor = numeros[0];

for (int numero : numeros)
{
   if (numero > mayor)
   {
      mayor = numero;
   }
}
```

Tiempo: O(n). Espacio: O(1), porque solo usamos una cantidad fija de variables adicionales.

15. O(n) espacio

```
vector<int> copia;

for (int numero : numeros)
{
   copia.push_back(numero);
}
```

Si numeros tiene n elementos, copia puede terminar con n elementos. Espacio: O(n).

16. O(n^2) espacio

```
vector<int> resultados;

for (int i = 0; i < numeros.size(); i++)
{
   for (int j = 0; j < numeros.size(); j++)
   {
      resultados.push_back(numeros[i] + numeros[j]);
   }
}
```

Se almacenan n x n resultados. Espacio: O(n^2).

17. Calcular no significa almacenar

```
cout << numeros[i] + numeros[j];
```

Este calculo puede ejecutarse muchas veces sin crear una estructura que crezca. Imprimir o calcular un valor no implica automaticamente mayor complejidad espacial.

18. Big O en vector

Operacion	Ejemplo	Complejidad
Acceso por indice	numeros[5]	O(1)
Buscar	find(...)	O(n)
Agregar al final	push_back(...)	O(1) amortizado
Insertar al principio	insert(begin(), ...)	O(n)

19. O(1) amortizado

push_back() normalmente agrega directamente al final en O(1). De vez en cuando el vector necesita reservar mas memoria y mover elementos, lo que puede costar O(n). Analizando muchas inserciones juntas, el costo promedio por insercion se considera O(1) amortizado.

Idea clave: O(1) amortizado describe tiempo por insercion. Un vector que almacena n elementos sigue usando O(n) espacio.

20. std::sort()

```
sort(numeros.begin(), numeros.end());
```

std::sort() tiene complejidad O(n log n), mucho mejor que un ordenamiento cuadratico como Bubble Sort para grandes cantidades de datos.

```
Bubble Sort -> O(n^2)
std::sort   -> O(n log n)
```

21. ¿Por que O(n log n)?

Conceptualmente, muchos algoritmos de ordenamiento eficientes realizan aproximadamente log n niveles de division/reduccion y trabajan con aproximadamente n elementos por nivel.

```
n trabajo por nivel x log n niveles = O(n log n)
```

22. map

```
buscar   -> O(log n)
insertar -> O(log n)
eliminar  -> O(log n)
```

Ventaja: mantiene las claves ordenadas.

23. unordered_map

```
Promedio:
buscar   -> O(1)
insertar -> O(1)
eliminar  -> O(1)

Peor caso teorico -> O(n)
```

No mantiene las claves ordenadas. El peor caso puede aparecer por colisiones de hash.

24. set

```
buscar   -> O(log n)
insertar -> O(log n)
eliminar  -> O(log n)
```

Mantiene elementos ordenados y no permite duplicados.

25. unordered_set

```
Promedio:
buscar   -> O(1)
insertar -> O(1)
eliminar  -> O(1)

Peor caso teorico -> O(n)
```

No mantiene orden y no permite duplicados.

26. Elegir estructura segun el uso

- Busqueda rapida, sin duplicados y sin importar el orden -> unordered_set.
- Relacion clave -> valor, busqueda rapida y sin importar orden -> unordered_map.
- Necesidad de mantener orden -> set o map.

27. Busqueda lineal

```
find(numeros.begin(), numeros.end(), buscado);
```

Peor caso: O(n). Puede necesitar revisar todos los elementos.

28. Busqueda binaria

```
binary_search(numeros.begin(), numeros.end(), buscado);
```

Complejidad: O(log n), pero requiere que los datos esten ordenados.

29. Ordenar antes de buscar

```
sort(...);          // O(n log n)
binary_search(...); // O(log n)
```

El costo total se simplifica a O(n log n). Si solo vas a hacer una busqueda sobre datos desordenados, find() con O(n) puede ser mas conveniente que ordenar primero.

30. Muchas busquedas

Si hacemos k busquedas con find():

```
O(k * n)
```

Si ordenamos una vez y hacemos busqueda binaria:

```
O(n log n + k log n)
```

Cuando hacemos muchas busquedas sobre los mismos datos, puede valer la pena ordenar primero.

31. No basta con mirar el ciclo

```
unordered_map<int, int> frecuencias;

for (int numero : numeros)
{
   frecuencias[numero]++;
}
```

El ciclo recorre n elementos, pero tambien debemos analizar el costo de frecuencias[numero]. En unordered_map es O(1) promedio por operacion, por lo que el algoritmo completo es O(n) promedio. En el peor caso teorico, si cada operacion se degrada a O(n), puede llegar a O(n^2).

32. Ejemplo con unordered_set

```
unordered_set<int> vistos;

for (int numero : numeros)
{
   if (vistos.find(numero) != vistos.end())
   {
      return true;
   }

   vistos.insert(numero);
}

Mejor caso:       O(1)
Promedio:         O(n)
Peor caso teorico: O(n^2)
Espacio:          O(n)
```

33. Reglas importantes para recordar

```
Acceso directo           -> O(1)
Un recorrido             -> O(n)
Dos recorridos separados  -> O(n)
Dos ciclos anidados       -> O(n^2)
Tres ciclos anidados      -> O(n^3)
Dividir entre 2 cada vez  -> O(log n)
n trabajos logaritmicos   -> O(n log n)
Crear n elementos nuevos  -> O(n) espacio
Crear n^2 elementos       -> O(n^2) espacio
```

34. Simplificaciones frecuentes

```
O(2n)       -> O(n)
O(n + 5)    -> O(n)
O(n^2 + n)  -> O(n^2)
O(n^3 + n^2) -> O(n^3)
O(n log n+n) -> O(n log n)
```

35. Idea principal del modulo

Antes bastaba con preguntarnos si una solucion funcionaba. Ahora tambien debemos evaluar cuanto trabajo realiza, cuanta memoria adicional usa, como escala al crecer la entrada y si existe una estructura o algoritmo mejor.

Conclusion: Una solucion puede funcionar bien con 10 elementos y volverse demasiado lenta con 1,000,000. Big O ayuda a escribir soluciones correctas y escalables.

Tabla rapida de consulta

Concepto	Complejidad / idea
Acceso vector[index]	O(1)
find(vector)	O(n)
binary_search (ordenado)	O(log n)
sort	O(n log n)
Bubble Sort	O(n^2)
map / set	O(log n)
unordered_map / unordered_set	O(1) promedio
push_back	O(1) amortizado
Vector auxiliar de n elementos	O(n) espacio
Matriz conceptual n x n almacenada	O(n^2) espacio